Reliable RL Scaling Requires Accounting for Prefill–Decode Kernel Mismatch

Yifan Zhang et al.

yifanzhangresearch@gmail.com

August 6, 2026*

Abstract

A language-model checkpoint does not by itself define the policy executed by a training system. The executable policy also depends on prompt-state construction, cached decoding, teacher-forced scoring, recurrent or chunkwise schedules, precision, quantization, distributed reductions, masks, and the sampling transform. Equal parameters can therefore induce different token distributions. Moreover, forward probability agreement at one parameter value is not sufficient to identify the gradient of a recurrent deployment policy: the trainer must differentiate the corresponding score function, or a forward-and-backward equivalent implementation.

We develop this execution-level view for softmax attention, linear attention, state-space models, and delta-rule memories. Prefill and decode form a composed behavior trace; parallel trainer scoring is a separate operator. For recurrent state models, a small discrepancy can enter later reads and writes and propagate through the remainder of the sequence. We distinguish this fixed-weight kernel gap from ordinary sampler-version staleness and specify the behavior-probability, target-score, support, record, and parity-audit contracts required to tell them apart.

We then compare four responses: preserve the sampler-emitted post-transform probability and expose the mismatch to any optimizer that consumes the record; make one chunkwise or recurrent rule canonical across sampler execution and differentiable trainer replay; promote recurrent states and sensitive updates to higher precision; or use a fast conditional proposal while a recurrent target performs exact modified rejection sampling. Precision is a mitigation rather than an equality proof, and architecture names do not establish execution parity. Together, these contracts make the executable policy auditable without assuming a particular policy-gradient algorithm. The validated evidence currently covers estimator sanity, fixed-weight production-path audits, forward-and-score mechanism tests, and a bounded support stress test. We exclude the previous undersized repeated-prompt RL pilot from learning claims and register a replacement scaled experiment before reporting end-to-end results.

Project Page: https://github.com/yifanzhang-pro/Pretraining-RL-Science

1 Introduction

A model checkpoint does not by itself determine the policy executed by a language-model system. The executable path also includes attention or recurrence kernels, prompt-state construction, cache

*Revised: August 24, 2026

layout, precision, quantization, parallel layout, reduction order, masks, and the sampling transform. Two engines with identical parameters can therefore define different token distributions.

Let K denote the complete execution operator relevant to one token decision. If $z_\theta^{\mathsf{K}}(h)$ is the logit vector at history h, define the effective policy

$$\pi_\theta^{\mathsf{K}}(\cdot \mid h) = \mathcal{T}_{\mathsf{K}}\left(z_\theta^{\mathsf{K}}(h)\right), \tag{1.1}$$

where $\mathcal{T}_{\mathsf{K}}$ includes temperature, admissibility masks, truncation, and any probability floor used by that path. A rollout record is distributionally on-policy for a trainer-defined target at parameters θ only if

$$\pi_\theta^{\mathsf{K}_{\text{sampler}}}(\cdot \mid h) = \pi_\theta^{\mathsf{K}_{\text{trainer}}}(\cdot \mid h) \tag{1.2}$$

for every history reached with positive sampler probability. Equal weights, checkpoint identifiers, token sequences, or model names do not imply Eq. (1.2).

There is a second, logically distinct contract. Define the score of an executable policy by

$$s_\theta^{\mathsf{K}}(a, h) := \nabla_\theta \log \pi_\theta^{\mathsf{K}}(a \mid h). \tag{1.3}$$

Forward equality in Eq. (1.2) makes the data on-policy for the declared trainer distribution at that fixed θ. It does not by itself show that a parallel trainer computes the gradient of a recurrent deployment policy. If the scientific target is $\pi_\theta^{\mathsf{K}_{\text{deploy}}}$, the trainer must differentiate $\log \pi_\theta^{\mathsf{K}_{\text{deploy}}}$, or use an implementation proved equivalent in both forward and backward computations. Equality of two forward distributions at one parameter value does not imply equality of their Jacobians.

A common implementation error is to sample with $\mathsf{K}_{\text{sampler}}$ but later reconstruct the "old" probability with $\mathsf{K}_{\text{trainer}}$. For a trainer-defined target, the recordwise sampled-token ratio is

$$\rho_t = \frac{\pi_{\theta_n}^{\mathsf{K}_{\text{trainer}}}(a_t \mid h_t)}{\pi_{\theta_v}^{\mathsf{K}_{\text{sampler}}}(a_t \mid h_t)}, \tag{1.4}$$

where v is the sampler version and n is the trainer version. The buggy recomputation instead uses

$$\widetilde{\rho}_t = \frac{\pi_{\theta_n}^{\mathsf{K}_{\text{trainer}}}(a_t \mid h_t)}{\pi_{\theta_v}^{\mathsf{K}_{\text{trainer}}}(a_t \mid h_t)}. \tag{1.5}$$

At zero parameter lag, $n = v$, Eq. (1.5) is one by construction, while Eq. (1.4) can still differ from one because the sampler and trainer executed different operators. Trainer-side recomputation therefore hides the kernel gap rather than measuring it.

This paper focuses on the executable-policy layer. It deliberately does not prescribe a particular policy-gradient objective, finite-step controller, or data-reuse scheme. That boundary is important: execution equivalence, support, and record validity are properties of the data-generating and scoring paths, and should be established independently of whichever optimization algorithm later consumes the records.

Our contributions are fourfold. First, we distinguish distributional parity from score-function parity and make both properties of an executable trace rather than a checkpoint. Second, we characterize the prefill–decode and recurrent–parallel gaps for softmax attention, linear attention, state-space models, and DeltaNet-like memories. Third, we separate fixed-weight kernel mismatch from sampler staleness and give auditable behavior-record and support contracts. Fourth, we compare alignment, precision, and proposal–verification remedies without treating any one of them as a universal proof of parity.

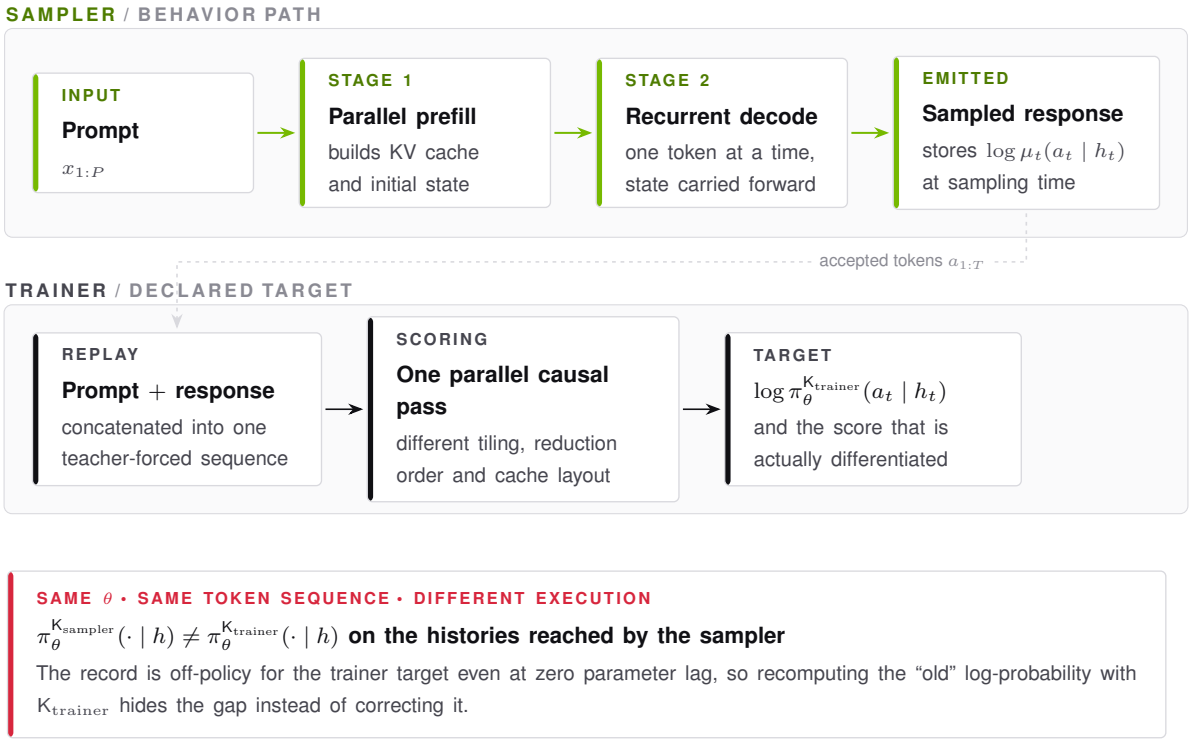


Figure 1 A common sampler–trainer split. The behavior path is a composed trace: prompt prefill constructs the initial cache or state, and recurrent decode extends it token by token. The trainer usually scores the same response with a single parallel teacher-forced pass, so the two paths define different executable policies at identical parameters.

2 Executable Policy Mismatch

2.1 Softmax Attention: Prefill and Decode

A standard autoregressive rollout has two phases. Prompt prefill constructs the KV cache and normally produces the distribution for the first generated token. Each later response token is produced by a one-token decode step whose state depends on that prefill cache and all preceding sampled tokens. The trainer usually concatenates the prompt and sampled response and evaluates them in one teacher-forced causal pass. Thus token 1 compares prompt-only prefill with a full-pass causal score, while tokens $t \geq 2$ compare a composed prefill–decode trace with that parallel pass. Either comparison can differ in finite precision, as shown in Fig. 1.

In exact real arithmetic, parallel causal softmax attention and the corresponding KV-cached recurrence can represent the same mathematical function. That algebraic identity is not a finite-precision equality certificate. Implementations can differ in

- tiling, fusion, and accumulation order;
- KV-cache layout and prompt-state construction;
- batch shape and sequence length;
- tensor- and sequence-parallel communication;

- precision, quantization, and reduction trees; and
- the post-logit sampling transform and support mask.

Any of these can perturb logits or change a truncation boundary. The resulting gap may be negligible in one configuration and large enough to destabilize RL in another, so it must be measured under the production configuration rather than inferred from checkpoint identity (Zhong et al., 2026; Zhang et al., 2025).

The sampled-token log-ratio can be decomposed into a fixed-weight execution gap and a sampler-version gap:

$$\log \rho_{t,n,v} = \underbrace{\log \pi_{\theta_n}^{\mathsf{K}_{\text{trainer}}}(a_t \mid h_t) - \log \pi_{\theta_n}^{\mathsf{K}_{\text{sampler}}}(a_t \mid h_t)}_{\Delta_{t,n}^{\text{kernel}}} + \underbrace{\log \pi_{\theta_n}^{\mathsf{K}_{\text{sampler}}}(a_t \mid h_t) - \log \pi_{\theta_v}^{\mathsf{K}_{\text{sampler}}}(a_t \mid h_t)}_{\Delta_{t,n,v}^{\text{stale}}}. \tag{2.1}$$

When $n = v$, the staleness term vanishes but the execution term need not. A fully synchronous pipeline can therefore still be off-policy for the trainer-defined target. Computing the two terms separately requires replaying the sampler operator at θ_n; only their sum is directly available from the stored sampler log-probability and the trainer score.

What contracts are sufficient? The strongest route is differentiable replay through the same prompt-state construction, KV-cache update, precision, masks, and tokenwise execution used by the sampler. The trainer must reconstruct the cache under its own parameters and propagate gradients through it; reusing a detached sampler cache omits parameter dependence through earlier states. A parallel path is also valid as the trainer-defined target. It is literally on-policy at fixed weights only when its forward distribution equals the sampler distribution; agreement within a stated tolerance supports only an approximate claim. To claim that the parallel path computes the gradient of recurrent deployment, one additionally needs backward or score-function equivalence, not only forward probability parity.

2.2 Linear Attention and State-Space Models

For linear attention and state-space models, the distinction is structural. A tokenwise implementation is naturally a state transition

$$(o_t, S_t) = \Phi_\theta(S_{t-1}, x_t), \tag{2.2}$$

whereas high-throughput training often uses a parallel scan, a convolutional dual, or a chunkwise factorization.

In common chunkwise formulations, the chunk size C spans two schedule endpoints:

$$C = 1 \iff \text{tokenwise recurrence}, \qquad C = L \iff \text{one full-sequence chunk}. \tag{2.3}$$

The $C = L$ endpoint may compute causal outputs in parallel inside the chunk; the exact schedule is architecture- and kernel-dependent. Intermediate chunk sizes trade sequential state propagation for matrix-multiplication throughput (Yang et al., 2023, 2024). Even when recurrent, scan, and chunkwise formulas are algebraically equivalent, changing the scan tree, chunk boundaries,

state materialization points, or accumulation precision changes finite-precision parenthesization. A discrepancy at one boundary can affect every later state and token probability.

A recurrent sampler paired with a chunkwise trainer must therefore be treated as two effective policies unless parity has been established for the exact chunk schedule, boundary convention, precision, parallel layout, masks, and prompt-state construction used in the run.

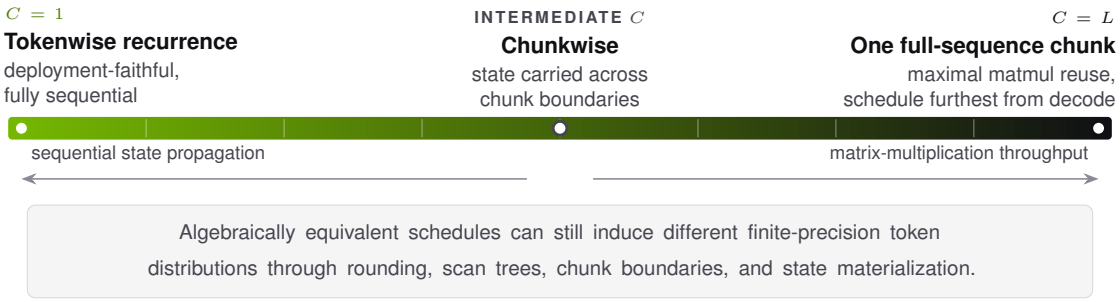


Figure 2 Chunk size selects an execution schedule, trading sequential state propagation for matrix-multiplication throughput. Algebraic equivalence does not by itself establish equality of the induced finite-precision token distributions.

2.3 DeltaNet as a Sensitive Case

DeltaNet (Schlag et al., 2021) makes schedule-dependent state error especially consequential because the state update is a data-dependent corrective write. In one common orientation,

$$S_t = S_{t-1}\left(I - \beta_t k_t k_t^\top\right) + \beta_t v_t k_t^\top, \tag{2.4}$$

which is equivalent to $S_t = S_{t-1} + \beta_t(v_t - S_{t-1}k_t)k_t^\top$. Common high-throughput training implementations rewrite the ordered recurrence using products of Householder-like factors and a memory-efficient chunkwise representation (Yang et al., 2024). The factors generally do not commute, so temporal order must be preserved algebraically. Even when it is preserved, the factorization changes floating-point parenthesization and the points at which intermediate values are rounded.

The discrepancy then enters future updates. An error in S_{t-1} changes the read $S_{t-1}k_t$, the erase residual $v_t - S_{t-1}k_t$, and hence the next state. State error is therefore not merely observed downstream; it participates in subsequent writes. Recurrent DeltaNet sampler and trainer scoring with a parallelized delta-rule kernel should be regarded as different effective policies unless forward and, when required, backward parity are demonstrated under the production configuration. The same warning applies to gated delta-rule variants and other recurrent architectures whose training kernel changes the update schedule.

3 Execution Contracts for Downstream RL

The previous section establishes that sampler and trainer paths can define different executable policies. A downstream RL method can respond coherently only after the system exposes which policy generated each token and which policy is being differentiated. This section defines that interface without committing to a particular estimator.

3.1 The sampler executes a time-dependent trace

The sampler does not use one isolated kernel for the entire response. For a softmax-attention sampler, a useful shorthand is

$$\mathsf{K}_{\mathrm{beh},t} = \begin{cases} \mathsf{K}_{\mathrm{prefill}}, & t = 1, \\ \mathsf{K}_{\mathrm{decode}}^{(t-1)} \circ \mathsf{K}_{\mathrm{prefill}}, & t \geq 2, \end{cases} \tag{3.1}$$

where $\mathsf{K}_{\mathrm{decode}}^{(t-1)}$ denotes the stateful trace through the preceding response tokens $a_{1:t-1}$. Thus the label "decode" includes the prompt state created by prefill and every earlier decode transition. For a linear-attention or state-space sampler, $\mathsf{K}_{\mathrm{beh},t}$ similarly denotes the complete recurrent, scan, or chunkwise trace that produced the state used at token t.

Let

$$\mu_{v,t}(\cdot \mid h_t) := \pi_{\theta_v}^{\mathsf{K}_{\mathrm{beh},t}}(\cdot \mid h_t) \tag{3.2}$$

be the post-transform behavior distribution at sampler version v, and let

$$p_{n,t}(\cdot \mid h_t) := \pi_{\theta_n}^{\mathsf{K}_{\mathrm{trainer}}}(\cdot \mid h_t) \tag{3.3}$$

be the trainer-defined target. The sampler stores $\ell_{t,v}^{\mu} = \log \mu_{v,t}(a_t \mid h_t)$ at sampling time. That value is immutable: a trainer replay may add diagnostics, but it may not replace the probability that actually generated the token.

3.2 Kernel mismatch and sampler staleness are separate

The total ratio in Eq. (1.4) factors into a fixed-weight execution term and a version term:

$$\rho_{t,n,v} = \underbrace{\frac{p_{n,t}(a_t \mid h_t)}{\pi_{\theta_n}^{\mathsf{K}_{\mathrm{beh},t}}(a_t \mid h_t)}}_{\rho_{t,n}^{\mathrm{kernel}}} \underbrace{\frac{\pi_{\theta_n}^{\mathsf{K}_{\mathrm{beh},t}}(a_t \mid h_t)}{\mu_{v,t}(a_t \mid h_t)}}_{\rho_{t,n,v}^{\mathrm{stale}}}. \tag{3.4}$$

The first factor can remain when $n = v$; the second can remain when sampler and trainer use the same operator but different versions. Their product is available from the stored sampler probability and the current trainer score. Separating the two factors requires replaying the sampler operator at current weights and is therefore best performed on a controlled audit subset. Logging only the total gap cannot identify whether sampler refresh, kernel alignment, or numerical changes are the appropriate intervention.

Intermediate bridge operators can further localize the execution term—for example, prompt-only prefill, composed prefill–decode, chunkwise replay, and tokenwise recurrence. Such factors are diagnostics only when they refer to exact intermediate distributions and telescope to the recorded total ratio. Multiplying independently defined "corrections" can double-count the same mismatch.

3.3 Support is part of the sampling contract

Any downstream likelihood ratio requires the declared target to be absolutely continuous with respect to the behavior distribution on admitted histories:

$$p_{n,t}(\cdot \mid h_t) \ll \mu_{v,t}(\cdot \mid h_t). \tag{3.5}$$

This is an executable sampling property, not something that sampled log-probabilities can establish after the fact. Hard top-k, top-p, min-p, or other logit-dependent truncation can assign zero behavior probability to an action that the trainer target retains, especially when a small kernel difference changes the cutoff ordering. A fixed grammar or safety mask is compatible with Eq. (3.5) only when sampler and trainer share the same admissible set. A shared full-support transform or a declared support-restricted target are coherent alternatives; silently pairing a truncated sampler with an untruncated trainer is not.

The record must therefore identify temperature, admissibility masks, truncation, probability floors, tokenizer, and every execution choice that can change support. Precision underflow to an exact zero is a support failure even when the mathematical softmax is positive.

3.4 Scope of the execution contract

The contracts above answer three execution questions: which distribution generated the action, which score function the trainer differentiates, and whether the target is supported by the behavior. They do not by themselves choose an RL objective, finite-step controller, or data-reuse schedule. This separation is intentional: a truthful behavior record and a declared target score are prerequisites for any execution-aware RL method, but they do not determine whether an optimizer uses token-local, prefix, or trajectory correction. Those choices must be stated separately by the optimization method being evaluated.

4 Aligning the Executable Policy

Once the mismatch has been exposed by the record contract, one option is to preserve distinct sampler and trainer policies for the downstream optimizer. The alternative studied here is to make one recurrence or chunk rule part of the model semantics and execute it consistently.

4.1 Canonical chunkwise updates

Let the accepted token stream be divided into blocks $X_j = x_{jC+1:(j+1)C}$. A semantic chunkwise state machine can be written as

$$O_j = G_\theta^{(C)}(S_j, X_j), \qquad S_{j+1} = F_\theta^{(C)}(S_j, X_j). \tag{4.1}$$

Literal alignment requires the sampler and trainer to share the same C, boundary convention, within-chunk causal mask, read-before-write rule, optimizer-like memory update, precision, and state-materialization points.

TTT layers define the hidden state as a model updated by a self-supervised learning step and use mini-batches of tokens plus a dual form for hardware efficiency (Sun et al., 2024). Titans likewise reformulates mini-batch memory updates with matmuls and parallel scans, and some variants segment the sequence explicitly (Behrouz et al., 2026). These works show that chunking can be part of model semantics rather than a hidden implementation detail. They do not imply that any two kernels called "TTT", "Titans", or "chunkwise" are equal. Changing chunk size, momentum carry, boundary resets, update timing, or scan tree defines a different executable policy unless equivalence is established.

EXECUTION CHOICE	REQUIRED CONTRACT	MAIN BENEFIT	MAIN COST / CAVEAT
Parallel target, recurrent sampler	Store the sampler probability and declare the trainer score	Highest trainer throughput	Requires an execution-aware optimizer and an explicit estimand
Canonical chunk rule	Same semantics, boundaries, precision, and backward	Chunk-level matmul efficiency	Unknown autoregressive tokens remain sequential
Pure recurrent replay	Same tokenwise path with gradients through state	Strongest deployment-gradient contract	Serial sequence replay and activation cost
Verified parallel implementation	Forward parity for policy; backward parity for deployment gradient	Parallelism without a fixed-weight kernel correction	Verification is configuration-specific

Table 1 Execution alignment trades throughput flexibility for stronger forward- and backward-equivalence contracts.

A chunkwise memory update may process a block of *already accepted* tokens together. It does not make unknown stochastic tokens available in advance. Exact parallel token generation still requires a proposal-and-verification mechanism, discussed in Sec. 4.4.

4.2 Pure recurrent training and inference

The strongest execution contract uses the same tokenwise recurrence everywhere:

$$S_t = F_\theta(S_{t-1}, x_t), \qquad z_t = G_\theta(S_t, x_t), \qquad a_t \sim \mathcal{T}(z_t). \tag{4.2}$$

Sampler execution, differentiable trainer replay, validation, and deployment execute Eq. (4.2) with the same state initialization, order, precision, and masks. The trainer reconstructs states under its own parameters and differentiates through them.

RWKV-7 is a useful architectural example because it provides constant-memory, constant-time recurrent inference per token while retaining parallelizable training (Peng et al., 2025). Therefore, "use RWKV-7" is not a parity certificate. The trainer must intentionally differentiate the recurrent form, or use a parallel implementation shown equivalent in both forward and backward computations.

Tokenwise replay sacrifices sequence-dimension parallelism and can be expensive for long responses. Activation checkpointing or exact recomputation can reduce memory without changing the mathematical gradient, provided the recomputation is deterministic under the declared numerical contract. Truncated backpropagation and state detachment do change the gradient contract. The benefit of recurrent replay is conceptual clarity: at fixed weights, there is no parallel–recurrent importance factor when the same recurrence defines both behavior and target.

4.3 Higher Precision for Recurrent State

Linear-attention and state-space layers feed a stored state into every subsequent update. To isolate numerical effects, consider a fixed teacher-forced input sequence and let S_t be a reference recurrence,

$\widehat{S}_t$ an implementation, and $e_t = \widehat{S}_t - S_t$. A generic perturbation bound is

$$\|e_t\| \le L_t \|e_{t-1}\| + \delta_t^{\text{round}} + \delta_t^{\text{schedule}}, \tag{4.3}$$

where L_t is a local state-sensitivity factor, δ_t^{round} is arithmetic error under a fixed schedule, and $\delta_t^{\text{schedule}}$ captures a changed scan tree, chunk boundary, or materialization rule. Unrolling gives

$$\begin{aligned} \|e_t\| \le & \left(\prod_{j=1}^{t} L_j \right) \|e_0\| \\ & + \sum_{u=1}^{t} \left(\prod_{j=u+1}^{t} L_j \right) \left(\delta_u^{\text{round}} + \delta_u^{\text{schedule}} \right). \end{aligned} \tag{4.4}$$

Higher precision can reduce δ^{round}. It does not remove schedule error, nor does it prevent amplification when products of the L_j are large.

A correctness-first precision ladder is:

1. **Establish an FP32 state baseline.** Keep recurrent states, decay products, normalization accumulators that feed the recurrence, and state-update reductions in FP32 while allowing unrelated embeddings, MLPs, and communication to remain in BF16 or FP16.

2. **Promote sensitive parameters and operands as needed.** Recurrence-defining gates, time constants, and update operands may also require FP32. The official Mamba implementation notes that storing main parameters in higher precision can be necessary because SSMs are sensitive to recurrent dynamics (Gu and Dao, 2023).

3. **Use FP64 as a reference, not a default training format.** FP64 recurrent states and updates are useful for small-model audits, sampled boundary checks, and diagnosing whether remaining error is numerical or semantic.

For DeltaNet-like updates, a useful minimum is to compute $S_{t-1}k_t$, the residual $v_t - S_{t-1}k_t$, and the rank-one accumulation in FP32. If discrepancies remain, promote the recurrent state and recurrence-defining operands to FP64 in the audit path. Casting an already rounded BF16 state to FP32 cannot recover lost information; the higher precision must be used before and during accumulation.

Higher precision is a mitigation, not an on-policy certificate. Two FP64 programs with different scan trees can still disagree, and hard truncation can turn a small logit perturbation into a support change. Precision experiments should therefore be paired with the probability, state, and score audits in Sec. A.1.

4.4 Fast Proposal with Recurrent Verification

A fourth route separates speed from policy semantics. Let the recurrent implementation define the canonical post-transform target distribution

$$p_i(\cdot) = \pi_\theta^{\mathsf{K}_{\text{recur}}}(\cdot \mid h_i), \tag{4.5}$$

Algorithm 1 Conditional proposal with recurrent target verification.

Require: Committed recurrent state S; proposal conditionals q_i; recurrent target $\mathsf{K}_{\mathrm{recur}}$; draft length γ.

Sample $y_{1:\gamma}$ from the proposal law in Eq. (4.6); retain every $q_i(\cdot \mid h_1, y_{<i})$.
From a temporary copy of S, run the recurrent target along the candidate prefix to obtain $p_1, \ldots, p_\gamma$ and temporary target states.
for $i = 1, \ldots, \gamma$ **do**
    Assert $q_i(y_i) > 0$ and draw $u_i \sim \mathrm{Uniform}(0, 1)$.
    if $u_i \leq \min(1, p_i(y_i)/q_i(y_i))$ **then**
        Commit y_i and the corresponding recurrent target state transition.
    else
        Draw $z \sim \mathrm{Normalize}([p_i - q_i]_+)$ from the state before position i.
        Commit z through the recurrent target; discard all later proposal and temporary states; **stop**.
    end if
end for
If every draft token is accepted, optionally sample one additional token from the next recurrent target distribution.

and let a faster operator define proposal conditionals q_i. The standard tokenwise modified-rejection algorithm is exact when the drafted block is actually sampled from

$$q(y_{1:\gamma} \mid h_1) = \prod_{i=1}^{\gamma} q_i(y_i \mid h_1, y_{<i}), \tag{4.6}$$

and the recorded q_i are those conditional distributions. The target verifier evaluates $p_i(\cdot \mid h_1, y_{<i})$ on the same candidate prefix.

For each proposed token, accept with probability

$$\alpha_i(y_i) = \min\left(1, \frac{p_i(y_i)}{q_i(y_i)}\right). \tag{4.7}$$

If token y_i is rejected, draw a replacement from

$$r_i(a) = \frac{\big[p_i(a) - q_i(a)\big]_+}{\sum_b \big[p_i(b) - q_i(b)\big]_+}. \tag{4.8}$$

Applied from left to right, this recovers the recurrent target distribution up to the declared hardware numerics (Leviathan et al., 2023; Chen et al., 2023). Argmax agreement, a logit tolerance, or arbitrary accept-then-resample logic does not generally preserve stochastic target sampling.

A teacher-forced prefill kernel can score a supplied candidate block, but it cannot by itself generate later autoregressive tokens because each position depends on the candidate prefix. A valid drafter may be a smaller autoregressive model, a same-model approximate recurrence, or another proposal mechanism whose actual conditional or joint law is available. For an arbitrary non-autoregressive or correlated block proposal, marginal token probabilities are not enough: one needs a joint/block correction or a specialized proof. Applying Eqs. (4.7) and (4.8) to convenient marginals is only a heuristic.

This arrangement reverses the usual performance emphasis. Standard speculative decoding often uses a cheap autoregressive drafter and a parallel target verifier. Here the recurrent implementation

is the correctness authority. The scheme helps only if proposal generation is cheap, acceptance is high, and target verification can exploit useful batching; it may provide no speedup when recurrent verification remains the dominant serial cost.

The proposal acceptance ratio is not the RL importance ratio used when an optimizer consumes off-policy records. After exact verification, committed tokens are distributed according to p_i, so the rollout record stores $\log p_i(a_i)$ as the behavior log-probability and treats proposal probabilities as metadata. If the trainer later declares a different parallel or chunkwise target, the recurrent sampler probability remains the behavior denominator. If the recurrent policy itself is the learning target, the trainer must differentiate the recurrent score; proposal correction does not replace the backward-equivalence contract.

5 Recommended Execution Contracts

A production system should choose one of two execution contracts rather than blur them.

1. **Recurrent-target mode.** The recurrent sampler and deployment operator define the target. The sampler stores recurrent post-transform probabilities, and the trainer differentiates recurrent replay or a forward-and-backward equivalent implementation. A fast path may propose tokens, but exact recurrent verification determines what is committed. When sampler and trainer versions match, this removes the fixed-weight kernel gap and is literally on-policy for the recurrent target.

2. **Declared parallel-target mode.** A recurrent sampler generates data, while a parallel or chunkwise trainer operator defines the score being differentiated. Every record preserves the sampler probability and execution signature. The two policies are intentionally distinct; the downstream optimizer must consume the actual behavior probability rather than reconstructing it with the trainer engine, and it must state explicitly which off-policy estimand it optimizes.

Both modes should use explicit support semantics, raw gap diagnostics, FP32 recurrent-state baselines, an FP64 audit path, and separate audits for forward distributions and score functions.

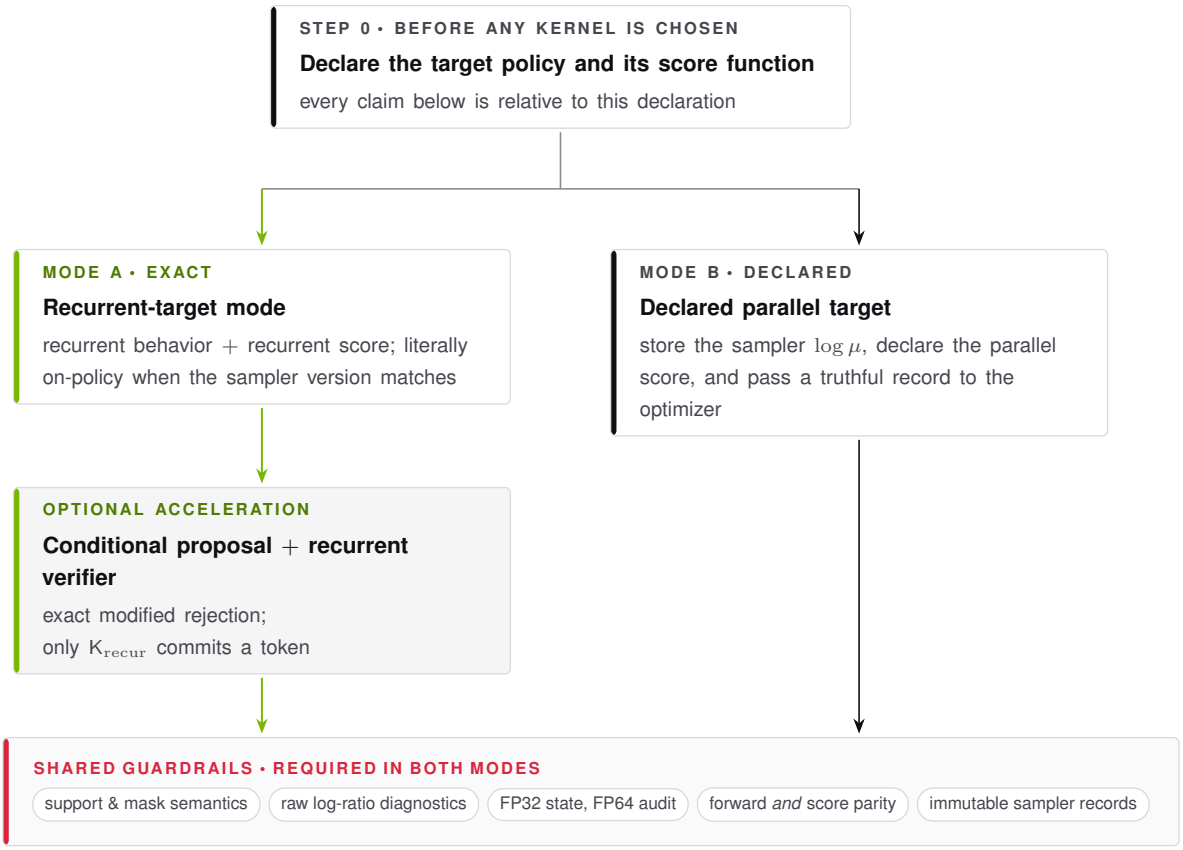


Figure 3 The target policy must be declared before kernels are chosen. Exact recurrent optimization and a declared parallel-target mode are both coherent, but they make different claims; the guardrails apply to either.

REMEDY	REQUIRED CONTRACT	WHAT IT ADDRESSES	RESIDUAL LIMITATION
Execution-aware records	Store the sampler's post-transform probability and complete execution signature	Makes the behavior law auditable and preserves the kernel gap	Does not by itself choose or stabilize an RL objective
Canonical execution rule	Same semantics, boundaries, precision, state construction, and backward implementation	Removes hidden operator ambiguity at fixed weights	May reduce sequence parallelism and throughput
FP32/FP64 recurrent state	Promote state and sensitive updates before accumulation	Reduces recurrent rounding drift and supplies a reference path	Cannot remove semantic schedule differences or support changes
Conditional proposal + recurrent verification	Actual proposal conditionals and exact modified rejection	Accelerates rollout without changing the recurrent target law	Needs high acceptance and may remain verification-bound

Table 2 The remedies act at the execution layer. They establish behavior and score contracts but do not by themselves select an objective, variance-reduction scheme, or trainer update rule.

6 Controlled Evidence

We evaluated the contracts above with an enumerable estimator check, a fixed-weight production-path audit, a forward-and-score mechanism test, and the bounded full-distribution stress test in the appendix. The distributed validated runs used exactly two H100 nodes: the production-path audit assigned one trainer node and one vLLM sampler node, the mechanism test used two condition shards, and the appendix audit used one capture node and one replay node. The retained runs pin model revisions, parent and Molt commits, data hashes, resolved configurations, node placement, and checksums. Qwen3-4B is a dense control and Qwen3.5-4B is a hybrid recurrent treatment; because they are not matched training runs or tokenizers, cross-model differences are descriptive. Causal method comparisons are within model.

Estimator and execution controls. In a vocabulary-4, horizon-6 environment, exhaustive enumeration verified the change-of-measure distinction. Full-trajectory IS matched the exact target gradient to 2.33×10^{-16}, whereas the exact raw token-local target remained 0.1551 away in ℓ_2 norm. At 200,000 trajectories over 30 seeds, their Monte Carlo errors to their own expectations were 0.00238 and 0.00143, respectively. Thus local IS is a valid estimator of the declared local surrogate, not of the full target-return gradient.

A fixed-weight production audit then selected the largest safe matched prompt, 8,192 tokens. Sampled-token $|\log \pi_{\text{learn}} - \log \pi_{\text{roll}}|$ p95 was 0.09293 for dense Qwen3, 0.04999 for Qwen3.5 with an FP16 recurrent cache, and 0.04433 with an FP32 cache. No cell had a nonfinite value or support violation. A separate 128-route gated-SSM mechanism test isolated the forward and score contracts. FP32 passed both: maximum state, logit, and gradient relative errors were 6.03×10^{-7}, 4.72×10^{-7}, and 6.18×10^{-7}. BF16 failed both, reaching 0.0175, 0.0131, and 0.0134, with up to 4.69% top-1 and 0.670% gradient-sign disagreement. At length 8,192, increasing scan chunk size from 1 to 256 raised FP32 throughput from 13.0K to 136.0K tokens/s while peak allocated memory rose from 86.9 to 100.5 MB. This exposes the fidelity–cost trade-off without treating precision as a proof of policy equality.

7 Registered Scaled RL Protocol

The earlier end-to-end pilot reused an undersized prompt set and is excluded from learning and method-comparison claims. Its replacement uses the English `math-dataset/DAPO-17k-Eng` source at an immutable revision. The source contains repeated accuracy-at-k rows, so the data gate first groups exact source identifiers, verifies repeat consistency, groups normalized prompts, and excludes all 128 prompts represented by multiple identifiers. This leaves 13,648 clean unique problems; a deterministic content-hash ordering assigns 12,800 to training and 848 to an internal holdout after requiring zero exact normalized-prompt collisions with the combined 60 prompts from pinned raw `math-ai/aime25` and `math-ai/aime26` evaluation inputs.

Evaluation uses the model-specific native-thinking profiles rather than the training sampler. Qwen3 uses temperature 0.6, top-p 0.95, top-k 20, min-p 0, no presence penalty, and repetition penalty 1. Qwen3.5 uses temperature 1, top-p 0.95, top-k 20, min-p 0, presence penalty 1.5, and repetition penalty 1. Both models use the same fixed 16,384-token output budget. A sample that reaches the cap scores as incorrect, and every cell reports its cap-hit rate; there is no per-cell fallback. The two benchmark scores remain visible separately rather than being replaced by the macro average.

REGISTERED QUANTITY	SCALED RL VALUE
Models	Immutable Qwen3-4B and Qwen3.5-4B revisions
Batch	128 unique prompt groups $\times$ 4 independent responses $= 512$ trajectories per optimizer step
Horizon	Two deterministic reshuffle epochs over 12,800 training problems; exactly 200 optimizer steps and 102,400 trajectory exposures
Length contract	At most 8,192 new response tokens and 12,288 total prompt-plus-response tokens; every admitted rendered prompt leaves 8,192 response tokens
Optimization	REINFORCE++ with whole-batch advantage whitening, reward 1 for correct and -0.1 otherwise, learning rate 10^{-6}, zero KL coefficient, and strict zero-lag synchronization
Mismatch arms	Qwen3: naive, raw token-local IS, and one-sided stabilized token-local IS; Qwen3.5: the same three plus an FP32 recurrent-state mitigation; seeds 17, 29, and 43
Topology	Two eight-H100 nodes per arm: one trainer node and one sampler node with four TP2 vLLM engines; interactive, ordinary batch, or reservation-backed allocations are valid, but matched arms keep one allocation lane and runtime
Checkpoints / evaluation	HF checkpoints every 20 steps through 200, plus resumable state; base and steps $20, 40, \ldots, 200$ each use 30 AIME25 and 30 AIME26 prompts with 32 native-thinking samples per prompt (1,920 samples total), report both benchmark-specific $\mathrm{avg@32} = \mathrm{pass1}$ values, and give their equal-weight macro average

Table 3 Registered scaled end-to-end RL protocol. Raw local IS uses the immutable sampler-emitted post-transform probability as its denominator; the stabilized arm caps only the upper token-weight tail at 1.15 while retaining raw-ratio diagnostics.

Each step must contain 128 distinct problem identifiers, 512 rollout records, and 512 trainer samples. Across the run, every training problem appears once per epoch. Sampler and trainer versions must match at every step; nonfinite values, support violations, missing checkpoint cadence, or unsafe resume lineage reject the arm. The naive seed-17 arm for each model and its complete checkpoint-evaluation matrix gate release of the remaining methods and seeds.

Promotion status. No scaled training arm, checkpoint evaluation, or end-to-end method comparison under this protocol is currently accepted. The table is a registration, not model-quality evidence; accuracy, learning, and method-comparison claims remain withheld until the immutable data, smoke, canary, checkpoint, and matched-matrix gates pass.

8 Conclusion

Prefill–decode mismatch is not resolved by synchronizing model weights. A standard sampler constructs state with prompt prefill and extends it with recurrent decode, while a trainer often scores the response with a parallel teacher-forced pass. Linear attention, state-space models, and delta-rule memories add schedule-dependent state evolution, so chunkwise, scan, and recurrent implementations can induce different finite-precision policies.

The central discipline is to separate three questions. Which post-transform distribution generated each token? Which executable policy and score function is the trainer differentiating? Which support and numerical contract makes that comparison meaningful? Sampler-emitted probabilities, execution signatures, and fixed-weight forward-and-backward audits answer these questions. Canonical recurrent or chunkwise replay can remove the gap when exact alignment is affordable. Higher precision reduces numerical drift but does not prove equality, and proposal–verification accelerates sampling only when the proposal law and recurrent acceptance rule are exact.

The validated results reinforce this execution-level claim. BF16 schedule changes altered both probabilities and scores in the mechanism test, and the production paths showed nonzero sampled-token gaps at zero sampler lag. Execution mismatch is therefore measurable and must be exposed to the downstream optimizer, but the magnitude of an audit metric alone does not establish a capability effect. The registered DAPO-17k scaled comparison will test whether naive, raw local-IS, stabilized local-IS, and higher-precision state produce different learning outcomes; no such outcome is claimed before its data, canary, checkpoint, and evaluation gates pass.

The checkpoint name is therefore not the policy. The policy is the executable distribution; an exact deployment-gradient claim additionally includes its score function. Once that interface is fixed, optimization claims can be evaluated without conflating them with execution semantics.

References

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *Advances in Neural Information Processing Systems*, 38:113506–113543, 2026.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*. PMLR, 2023.

Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Xingjian Du, Haowen Hou, Jiaju Lin, Jiaxing Liu, Janna Lu, William Merrill, et al. Rwkv-7" goose" with expressive dynamic state evolution. *arXiv preprint arXiv:2503.14456*, 2025.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, pages 9355–9366. PMLR, 2021.

Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.

Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. *arXiv preprint arXiv:2312.06635*, 2023.

Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2024.

Ziyang Zhang, Xinheng Ding, Jiayi Yuan, Rixin Liu, Huizi Mao, Jiarong Xing, and Zirui Liu. Deterministic inference across tensor parallel sizes that eliminates training–inference mismatch. *arXiv preprint arXiv:2511.17826*, 2025.

Tianle Zhong, Neiwen Ling, Yifan Pi, Zijun Wei, Tianshu Yu, Geoffrey Fox, Peng Wu, and Xiao Yu. Diagnosing training–inference mismatch in LLM reinforcement learning. *arXiv preprint arXiv:2605.14220*, 2026.

Appendix

A Auditing and Record Contracts **18**
A.1 Minimal Fixed-Weight Parity Audit 18
A.2 Bounded Full-Distribution Support Stress 18
A.3 Record and Logging Checklist 19
A.4 Minimal Execution Record Contract 20

B Decision Checklist **21**

A Auditing and Record Contracts

A.1 Minimal Fixed-Weight Parity Audit

A parity audit should be run independently of RL optimization. Freeze parameters, feed identical policy-visible prefixes to the actual sampler and trainer paths, and separate four quantities.

1. **State parity:** compare prompt states, cache tensors, and recurrent boundary states before comparing logits.
2. **Forward-policy parity:** compare post-transform token distributions, including masks, temperature, truncation, and probability floors.
3. **Score parity:** for selected token log-probabilities, compare gradients, vector–Jacobian products, or finite-difference directional derivatives. Forward equality at one checkpoint does not establish this contract.
4. **Sampling parity:** verify that the probability recorded for the sampled token is exactly the probability used by the sampler after all transforms.

ARCHITECTURE	EXECUTABLE COMPARISONS
Softmax attention	Prompt prefill, recurrent KV-cached decode, differentiable recurrent replay, and full teacher-forced causal scoring.
Linear attention / SSM	Recurrent, scan, and chunkwise forms across chunk sizes, boundaries, state-materialization schedules, and precision.
DeltaNet	Tokenwise delta-rule updates versus the production chunkwise kernel, including FP32 and FP64 state references.
TTT / Titans-style memory	Online updates versus the exact mini-batch or segment rule, with identical read/write timing, momentum, decay, and resets.
Speculative rollout	Actual proposal conditionals, recurrent verifier distributions, acceptance draws, residual samples, committed states, and final sampler probabilities.

Table 4 The audit compares executable traces and derivatives, not checkpoint names.

A.2 Bounded Full-Distribution Support Stress

We captured the full vLLM vocabulary distribution at four decode positions for each of four frozen prefixes and replayed the same 16 histories with a BF16 Transformers SDPA reference. Capture used TP=2 on one node and replay used the second node. This is an auxiliary full-distribution audit, not the Transformer-Engine trainer path used in the sampled-token experiment.

MODEL / CACHE	TV MEAN / P95	TOP-1 AGREE	TOP-20 JACCARD	TARGET MASS OUT
Qwen3 dense	7.01×10^{-6} / 3.90×10^{-5}	1.0	0.9129	3.90×10^{-6}
Qwen3.5 FP16	2.493×10^{-3} / 1.326×10^{-2}	1.0	0.9594	2.55×10^{-5}
Qwen3.5 FP32	2.391×10^{-3} / 1.320×10^{-2}	1.0	0.9702	2.54×10^{-5}

Table 5 E4 full-vocabulary results over 16 histories per arm. "Target mass out" is the mean target probability outside the sampler's counterfactual top-20 support.

All top-1 decisions agreed, yet the distributions and hard supports were not identical. Qwen3.5 mean TV was roughly $356\times$ the dense value with an FP16 cache and $341\times$ with FP32. FP32 reduced mean TV by 4.11% and improved top-20 Jaccard, but barely changed target mass outside sampler support. That missing mass was small on these bounded histories. We therefore retain FP32 as a numerical audit and mitigation option, not as an empirically accuracy-improving recommendation. Following the predeclared E4 disposition, this bounded result remains appendix evidence because support disagreement was not a dominant production failure mode.

Vary precision, quantization, batch shape, tensor parallelism, sequence parallelism, and chunk size one factor at a time. When full distributions are available, report both directions of KL where finite, total variation, maximum and mean log-probability error, support-set disagreement, and sampled ratio quantiles. For backward checks, report relative gradient or VJP error on representative parameters and recurrent states.

A finite audit can falsify equality and quantify a tested configuration; it cannot prove equality over all histories. Any exact "on-policy" or "equivalent" claim should therefore state the execution contract and justify why equality extends beyond the tested prefixes. A tolerance-based audit supports a configuration-specific approximate claim and should name the tolerance and whether it covers forward distributions, score functions, or both.

A.3 Record and Logging Checklist

For every sampled policy token, preserve the sampler-path post-transform log-probability and enough metadata to identify the behavior trace. At minimum, this includes sampler version, sampler engine, attention or recurrence form, prefill/decode trace, chunk size or scan schedule, cache construction, precision, quantization, masks, sampling transform, and tensor/sequence-parallel layout.

The total log-ratio

$$\Delta^{\text{total}}_{t,n,v} = \log \pi^{\mathsf{K}_{\text{trainer}}}_{\theta_n}(a_t \mid h_t) - \log \pi^{\mathsf{K}_{\text{sampler}}}_{\theta_v}(a_t \mid h_t) \tag{A.1}$$

is available from the trainer score and immutable sampler record. Its decomposition is

$$\Delta^{\text{kernel}}_{t,n} = \log \pi^{\mathsf{K}_{\text{trainer}}}_{\theta_n}(a_t \mid h_t) - \log \pi^{\mathsf{K}_{\text{sampler}}}_{\theta_n}(a_t \mid h_t), \tag{A.2}$$

$$\Delta^{\text{stale}}_{t,n,v} = \log \pi^{\mathsf{K}_{\text{sampler}}}_{\theta_n}(a_t \mid h_t) - \log \pi^{\mathsf{K}_{\text{sampler}}}_{\theta_v}(a_t \mid h_t). \tag{A.3}$$

Computing these two terms requires replaying the sampler operator at current weights and is usually best done on a sampled audit subset. Logging only Δ^{total} cannot identify whether refresh, kernel alignment, or precision is the appropriate intervention.

Exact per-history KL or TV requires either stored behavior distributions or a retained sampler checkpoint that can reproduce them under the recorded operator. Sampler log-ratios observed on sampled actions can estimate behavior-averaged forward KL, but a single action is not an exact per-history divergence and does not determine TV or the reverse KL.

A.4 Minimal Execution Record Contract

A rollout record is valid only if the trainer can identify the distribution that produced every policy token. At minimum it contains:

1. the exact policy-visible prompt and history, sampled token IDs, termination markers, and policy-token mask;
2. the sampler-emitted post-transform log-probability ℓ_t^μ for every policy token and the sampler version v;
3. a per-token or segment-resolved execution signature: prefill/decode trace, recurrent or chunkwise form, chunk boundaries, state and parameter precision, quantization, cache construction, parallel layout, and reduction mode;
4. sampling metadata: temperature, admissible-action mask identity, probability floor, grammar or safety constraints, and any top-k/top-p/min-p truncation or stored support mask;
5. enough model provenance to reproduce audit distributions, such as a retained sampler checkpoint or immutable content hash plus executable version;
6. for speculative rollout, the proposal law or actual conditionals q_i, verifier version, acceptance/rejection draws, residual sample, and final recurrent behavior probability $p_i(a_i)$; and
7. immutable reward, environment, verifier, termination, and sample-provenance metadata.

The trainer may append diagnostics but must never replace ℓ_t^μ with a trainer-side recomputation.

B Decision Checklist

QUESTION	REQUIRED ACTION
Is recurrent deployment the scientific target?	Differentiate recurrent replay or a verified forward-and-backward equivalent implementation; a recurrent ratio with a parallel score is not exact.
Do sampler and trainer match only in forward probabilities?	The data may be on-policy for the trainer distribution, but do not claim recurrent deployment-gradient equivalence without a score audit.
Is one chunk rule part of model semantics?	Freeze chunk size, boundaries, masks, read/write timing, momentum, decay, resets, precision, and backward implementation.
Must the trainer retain a faster parallel target?	Preserve sampler probabilities and support metadata, declare the parallel score, and require the downstream optimizer to consume the actual behavior law.
Does sampling use hard top-k/top-p/min-p?	Share and freeze an admissible support or label the run support-truncated; no ratio repairs actions with zero behavior probability.
Are recurrent states drifting?	Establish FP32 state/update arithmetic, compare against FP64, and separate rounding error from schedule error.
Can a fast path propose accurate blocks?	Use actual proposal conditionals with exact modified rejection and commit only recurrent target states.
Is the proposal non-autoregressive or correlated across positions?	Use a joint/block correction or specialized proof; token marginals are insufficient for standard speculative rejection.
Is the sampler–trainer probability gap large?	Separate kernel and version components, improve parity or refresh samplers, and expose any record rejection to the downstream data law.

Table 6 A compact decision tree for choosing among the execution contracts.